

WEB APPLICATION DEVELOPMENT USING FLASK

Milestone 1 Project Documentation

Submitted By

Name: Bhargavi Ketha

Course: Bachelor of Technology (B.Tech)

Department: Computer Science and Engineering

Project Title

FoodExpress – Web Application using Flask

Academic Year

2025 – 2026

Submission Date

25-01-2026

Abstract

This project demonstrates the development of a web application using the Flask framework in Python. Milestone 1 focuses on establishing the backend structure, implementing routing, designing HTML templates, and validating the application using local data storage. The objective is to ensure proper functioning of the application in a local environment before integrating cloud services and databases in future milestones.

Milestone 1 – Web Application Development and Setup

1. Introduction

This project involves the development of a web application using the **Flask framework** in Python. Milestone 1 focuses on building the foundational backend structure, defining routes, creating HTML templates, and validating application functionality using local data storage.

The main goal of this milestone is to ensure that the application works correctly in a local development environment before integrating databases and cloud services in future milestones.

2. Objectives of Milestone 1

The primary objectives of this milestone are:

- To initialize a Flask backend application
 - To design basic web pages using HTML templates
 - To implement routing and request handling in Flask
 - To store and retrieve user data using Python lists and dictionaries
 - To test the application locally using Flask's development server
-

3. System Architecture

The application follows a **Client–Server Architecture**:

1. The user interacts with the web application through a browser.
2. HTTP requests are sent to the Flask backend.
3. Flask processes requests, performs operations on local data structures.
4. Flask renders HTML templates and sends responses back to the browser.

This architecture allows testing of complete application logic without the need for external databases or cloud platforms.

4. Technology Stack

Component	Technology Used
Programming Language	Python
Backend Framework	Flask
Frontend	HTML, CSS
Data Storage	Python Lists and Dictionaries
Server Environment	Localhost (Flask Development Server)

5. Project Structure

```
milestone1_app/
|
|— app.py
|— requirements.txt
|
|— templates/
|   |— index.html
|   |— register.html
|   |— login.html
|   |— dashboard.html
```

File Description

- **app.py** – Contains the main Flask application logic and routes
 - **templates/** – Stores all HTML files rendered by Flask
 - **requirements.txt** – Lists the required Python packages
-

6. Implementation Details

6.1 Flask Application Setup

The Flask application is initialized in `app.py`. A secret key is configured to manage user session securely. The application runs using Flask's built-in development server on localhost.

6.2 Routing and Templates

The following routes are implemented:

Route	Description
/	Home page
/register	User registration page
/login	User login page
/dashboard	Dashboard after successful login
/logout	Ends user session

Each route renders a corresponding HTML template using Flask's `render_template()` function.

6.3 Local Data Storage

User data is temporarily stored using a **Python list of dictionaries**. This enables testing of registration and login functionality without using a database.

Example structure:

```
users = [{"username": "admin", "password": "1234"}]
```

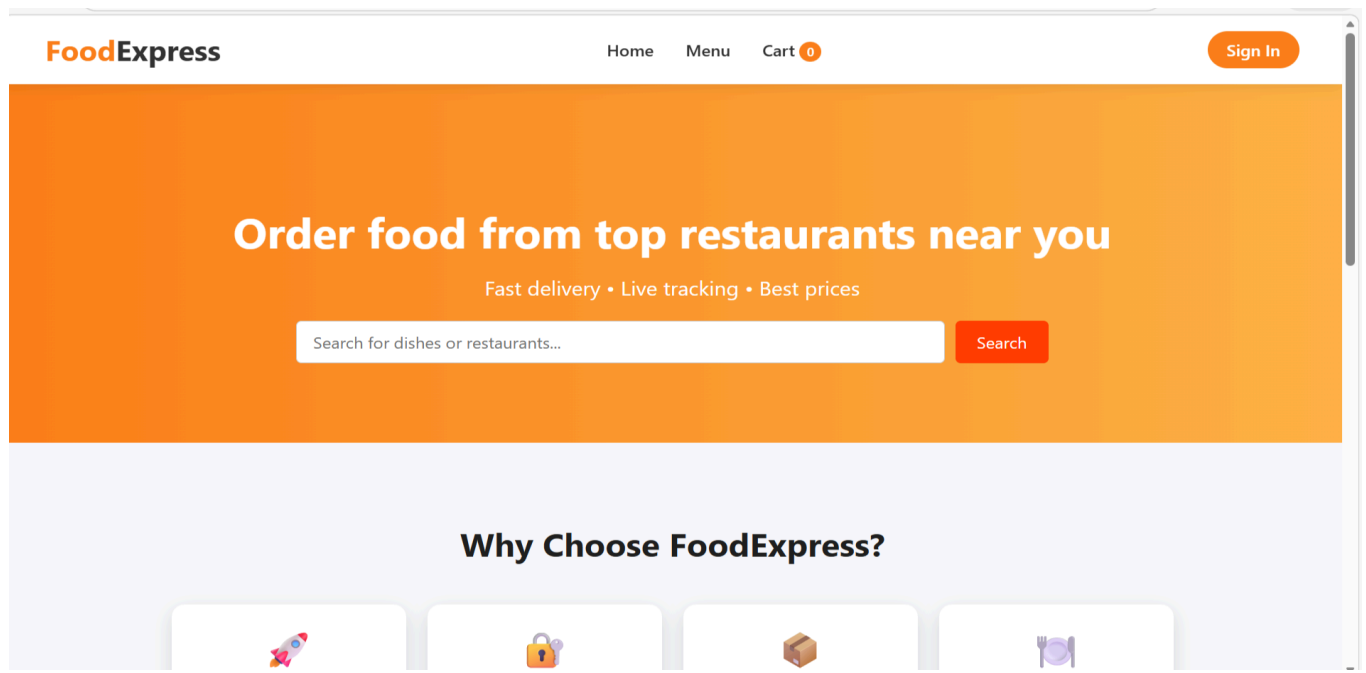
6.4 Authentication and Session Management

- User credentials are validated against the local data list.
 - Flask sessions are used to keep users logged in.
 - Unauthorized users cannot access the dashboard page.
 - Logout clears the session securely.
-

7. Application Output

The application was tested successfully on localhost and produced the following results:

- Flask server runs correctly on `http://127.0.0.1:5000`
- Users can register through the web interface
- Registered users can log in
- Authenticated users are redirected to the dashboard
- Session handling restricts unauthorized access



8. Limitations

- Data is stored in memory and lost when the server restarts
- No database integration in this milestone
- Security mechanisms are basic and for development purposes only

9. Conclusion

Milestone 1 successfully establishes the core structure of the web application using Flask. Routing, template rendering, session management, and local data handling are implemented effectively. The application runs smoothly in a local environment and satisfies all milestone requirements.

This milestone provides a strong foundation for future enhancements involving databases and cloud integration.

10. Future Scope

In the upcoming milestones, the following improvements will be implemented:

- Integration with AWS DynamoDB
 - Advanced authentication and security mechanisms
 - Deployment to a cloud environment
 - Improved scalability and performance
-

Milestone 1 Status: Completed Successfully 